Московский физико-технический институт
Высшая школа программной инженерии

Формальные языки и сложность вычислений
3 семестр, 2026/2027 уч. год

Лекция 6
NP-полнота задачи EXACTSETCOVER. Не NP-полные вариации задачи о рюкзаке. Теорема Махэни. Полиномиальная иерархия

Лектор: Зухба Анастасия Викторовна <3
Автор: Соля

1 NP-полнота задачи EXACTSETCOVER

Рассмотрим следующие три NP-полные задачки:

$$\text{EXACTSETCOVER} = \{(n, S_1, \ldots, S_m) \mid \text{можно выбрать непересекающиеся множества}$$
$$S_{i_1}, \ldots, S_{i_k}, \text{ в объединении дающие все числа от 1 до } n\}$$
$$\text{SUBSET-SUM} = \{(k_1, k_2, \ldots, k_m, N) \mid \exists a \in \{0,1\}^n \ \sum_{i=1}^{m} a_i k_i = N\}$$
$$\text{KNAPSACK} = \{((w_1, \ldots, w_n), (v_1, \ldots, v_n), W, V) \mid x_i \in \{0,1\}, \ \sum_{i=1}^{n} x_i w_i \le W, \ \sum_{i=1}^{n} x_i v_i \ge V\}$$

Замечание 1.1. *Задача о рюкзаке имеет множество различных постановок. Многие из них (но не все) являются* NP*-полными.*

Доказательство. Докажем NP-полноту задачи EXACTSETCOVER с помощью полиномиальной сводимости из задачи 3-SAT (принадлежность EXACTSETCOVER к NP очевидна, так как просто берём сертификат и жизнь сразу налаживается). Пусть дана формула

$$\varphi = \underbrace{(x \vee \overline{y} \vee z)}_{A} \wedge \underbrace{(\overline{x} \vee z \vee \overline{t})}_{B} \wedge \underbrace{(\ldots)}_{C} \wedge \ldots$$

в 3-КНФ. Мы хотим свести её к задаче EXACTSETCOVER. Дадим каждому дизъюнкту "имя" $(A, B, C, \ldots)$. Тогда множество, которое мы хотим получить в задаче EXACTSETCOVER, будет включать в себя

- x, x_0, x_1 для каждой переменной x
- C, C_1, C_2, C_3 для каждого дизъюнкта C

Из каких множеств мы будем его составлять? Сначала введём "заглушки":

- $\{x, x_0\}$, $\{x, x_1\}$ для каждой переменной x
- $\{C\}$, $\{C, C_1\}$, $\{C, C_2\}$, $\{C, C_3\}$, $\{C, C_1, C_2\}$, $\{C, C_1, C_3\}$, $\{C, C_2, C_3\}$ для каждого дизъюнкта C

Далее введём множества, которые могут сначала показаться контринтуитивными (захочется поменять местами 0 и 1), но на самом деле всё окей:

- $\{x_0, C_j, \ldots\}$ для каждой переменной x, если x встречается в дизъюнкте C в позиции j **без отрицания**
- $\{x_1, D_k, \ldots\}$ для каждой переменной x, если x встречается в дизъюнкте D в позиции k **с отрицанием**

Если φ выполнима, то возьмём выполняющий набор. Для каждой переменной x будем брать

- $\{x_0, C_j, \ldots\}$ и "заглушку" $\{x, x_1\}$, если x в выполняющем наборе равна 1
- $\{x_1, D_k, \ldots\}$ и "заглушку" $\{x, x_0\}$, если x в выполняющем наборе равна 0

После этого для каждого дизъюнкта C возьмём подходящую "заглушку", в которой будет C и все не вошедшие в объединение C_j. Тут важно заметить, что заглушка $\{C, C_1, C_2, C_3\}$ действительно не понадобится и всё пройдёт корректно, так как в противном случае делаем вывод о том, что ни одна переменная не "заединичила" дизъюнкт C, что противоречит тому, что мы взяли выполняющий набор. Посмотрим же, почему тогда мы решили задачу EXACTSETCOVER:

1. По построению мы охватили все элементы x, x_0, x_1

2. По построению мы охватили все элементы C, C_1, C_2, C_3

3. Ни один элемент переменной не был учтён дважды (также очевидно из построения)

4. Ни один элемент дизъюнкта не был учтён дважды, о чём давайте далее чуть подробнее

Почему последнее верно? Давайте просто заметим, что C_j по построению возникнет только тогда, когда x встретится в дизъюнкте C в позиции j (с отрицанием или без). Мы для каждой переменной x берём только один набор вида $\{x_\alpha, C_j, \ldots\}$. Если мы получили C_j дважды, то нашлись наборы $\{x_\alpha, C_j, \ldots\}$ и $\{y_\beta, C_j, \ldots\}$. Но ведь в дизъюнкте C не может быть двух разных переменных в одной позиции, значит $x = y$. Но тогда мы взяли два набора для одной переменной, что невозможно. Ура, уже доказано

$$\varphi \in \text{3-SAT} \implies f(\varphi) \in \text{EXACTSETCOVER}$$

В обратную сторону всё даже легче. У нас есть какая-то 3-КНФ φ и построенный по ней $f(\varphi)$ — набор множеств, который даёт решение задачи EXACTSETCOVER. Рассмотрим тогда это точное покрытие. Для каждой переменной x оно содержит либо $\{x, x_0\}$, либо $\{x, x_1\}$, так как x есть только там. Тогда оставшаяся x_α покрывается набором вида $\{x_\alpha, C_j, \ldots\}$. На самом деле уже на этом шаге мы для каждого дизъюнкта C покрыли один из его элементов C_j. Ведь иначе нам бы пришлось покрыть $\{C, C_1, C_2, C_3\}$, что невозможно сделать "заглушками" (одной большой нет, а между собой они пересекаются как минимум по C). Тогда просто посмотрим на попавшие в покрытие наборы вида $\{x, x_\alpha\}$ и положим $x = \alpha$ для каждой переменной x. Из рассуждений выше как раз становится ясно, что полученный набор значений переменных будет выполняющим для φ. Ура, доказали и обратное:

$$f(\varphi) \in \text{EXACTSETCOVER} \implies \varphi \in \text{3-SAT}$$

Осталось только пояснить, почему f — полиномиально вычислимая по длине φ. Действительно, количество переменных — полином от длины φ, количество дизъюнктов — полином от длины φ. Всё строится за полином от этих количеств, значит за полином от длины φ. Ура, доказали NP-полноту задачи EXACTSETCOVER. □

2 NP-полнота задачи SUBSET-SUM

Доказательство. Докажем NP-полноту задачи SUBSET-SUM с помощью полиномиальной сводимости из задачи EXACTSETCOVER. Пусть у нас есть экземпляр задачи EXACTSETCOVER — $\{n, S_1, S_2, \ldots, S_m\}$. Тогда в $(m+1)$-ричной системе счисления построим числа $k_1, k_2, \ldots, k_m$ и N следующим образом:

- k_i — число, у которого в j-той цифре стоит 1, если $j \in S_i$, и 0 иначе
- N — число из n единиц

Тогда очевидно, что если мы можем выбрать непересекающиеся множества $S_{i_1}, \ldots, S_{i_h}$, то сумма соответствующих чисел $k_{i_1} + \ldots + k_{i_h}$ будет равна N. В обратную сторону, если мы можем выбрать числа $k_{i_1}, \ldots, k_{i_h}$, сумма которых равна N, то в каждой цифре суммы будет стоять 1, что означает, что выбранные множества $S_{i_1}, \ldots, S_{i_h}$ непересекающиеся и дают в объединении все числа от 1 до n. Ура, доказали NP-полноту задачи SUBSET-SUM (полиномиальная вычислимость функции f очевидна, так как мы просто строим числа в $(m+1)$-ричной системе счисления). □

3 Задача о рюкзаке

Доказательство. Докажем NP-полноту задачи KNAPSACK с помощью полиномиальной сводимости из задачи SUBSET-SUM. Пусть у нас есть экземпляр задачи SUBSET-SUM — $\{k_1, k_2, \ldots, k_m, N\}$. Тогда просто положим $w_i = v_i = k_i$, $W = V = N$. Тогда очевидно, что если мы можем выбрать числа $k_{i_1}, \ldots, k_{i_h}$, сумма которых равна N, то мы можем положить в рюкзак предметы с весами и ценностями $w_{i_1}, \ldots, w_{i_h}$ и получить суммарный вес и ценность равные N. В обратную сторону, если мы можем положить в рюкзак предметы с весами и ценностями $w_{i_1}, \ldots, w_{i_h}$ и получить суммарный вес и ценность равные N, то мы можем выбрать числа $k_{i_1}, \ldots, k_{i_h}$, сумма которых равна N. Ура, доказали NP-полноту задачи KNAPSACK (полиномиальная вычислимость функции f очевидна, так как мы просто копируем числа). □

3.1 Задача о непрерывном рюкзаке

На самом деле не все вариации задачи о рюкзаке являются NP-полными. Например, если разрешить дробные значения x_i, то задача становится полиномиально решаемой. Действительно, в этом случае можно просто отсортировать предметы по убыванию ценности на единицу веса и брать их (опционально дробно) в рюкзак до тех пор, пока не достигнем ограничения по весу. Если набрали нужную ценность, то задача решена.

3.2 А что если W записана в унарной системе счисления?

Если W записана в унарной системе счисления, то в этом случае задача остается полиномиально решаемой, поскольку можно использовать динамическое программирование:

Algorithm 1 Задача о рюкзаке

```
dp[0..W] ← 0
for все предметы (w, v) do
    for j = W downto w do
        dp[j] ← max(dp[j], dp[j − w] + v)
    end for
end for
return dp[W]
```

Получили сложность $O(nW)$ и доп память $O(W)$.

4 Теорема Махэни

Определение 4.1. ***Разреженный язык*** *— это формальный язык, для которого количество слов длины n ограничено полиномом от n.*

Теорема 4.1 (Махэни)**.** *Если существует язык, который одновременно является* NP*-полным и разреженным, то* P = NP*.*

Лемма 4.1. *Любой унарный язык является разреженным.*

Следствие 4.1. *Если существует унарный* NP*-полный язык, то* P = NP*.*

Замечание 4.1. *Следствие выше как раз и позволяет нам понять, что перекодировать* NP*-полный язык в унарную запись — не супер идея.*

Доказательство. Пусть S — NP-полный и разреженный язык. Так как он NP-полный, то SAT полиномиально сводится к нему, т.е.

$$\exists f \text{ — всюду определенная полиномиально вычислимая функция: } x \in \text{SAT} \iff f(x) \in S$$

Покажем тогда, как решить SAT за полиномиальное время:

У нас точно будет монотонный полином $p(m)$, ограничивающий количество слов длины не более m (изначально в определении сказано ровно про m и нет ничего про монотонность, но первое очевидно следует из того, что есть полином, ограничивающий количество слов длины ровно m, и что мы можем просто взять его максимум на отрезке $[0, m]$ и домножить на m, а второе следует из того, что любой не монотонный полином можно сверху ограничить монотонным).

Теперь рассмотрим формулу φ длины N и в ней n переменных. При подстановке некоторых значений переменных мы получаем формулу $\widetilde{\varphi}$ длины не более $c \cdot (N+1)$, где c — константа. Подставляя по одному значению переменной, получаем каждый раз увеличение кол-ва формул в 2 раза. Мы знаем, что выполнимость φ на самом деле равносильна выполнимости хотя бы одной из формул Θ_i, полученных после подстановки. Пока что нас конечно не радует, что их 2^n, поэтому посмотрим, как их отсекать без потери информации.

У нас сейчас есть список формул $\Theta_1, \ldots, \Theta_k$, каждая из которых имеет длину не более $c \cdot (N+1)$. Если взять дизъюнкцию двух из них $\Theta_i \vee \Theta_j$, то она будет иметь длину не более $c' \cdot (N+1)$. Теперь вспомним, что f — полиномиально вычислимая функция $\implies f(\Theta_i \vee \Theta_j)$ будет ограничена сверху полиномом по N. Пусть этот полином — $R(N)$. Тогда введём $T(N) = p(R(N))$. Получается, что среди S не более $T(N)$ слов длины не более $R(N)$.

Пока $k > T(N) + 1$ и мы ещё не подставили все значения переменных, подставляем очередное значение переменной, после чего берём любую формулу (например Θ_1) и дизъюнкцию её со всеми другими формулами. Положим

$$z_i = f(\Theta_1 \vee \Theta_i)$$

Случай 1

$$\exists i \neq j : \; z_i = z_j$$

Рассмотрим тогда формулы $\Theta_1 \vee \Theta_i$ и $\Theta_1 \vee \Theta_j$. Так как f — всюду определённая функция, то $z_i = z_j \implies f(\Theta_1 \vee \Theta_i) \in S \iff f(\Theta_1 \vee \Theta_j) \in S$. Тогда мы можем просто удалить из списка формулу Θ_j, так как она не несёт никакой новой информации.

Случай 2

$$\forall i \neq j : \; z_i \neq z_j$$

Тогда из разреженности языка S следует, что $\exists i : z_i \notin S \implies \Theta_1$ не выполнима и её можно удалить из списка.

Так мы каждый раз удаляем, пока не вернём $k \leq T(N) + 1$. Затем просто подставляем очередное значение переменной (если ещё не завершили), получая $k \leq 2T(N) + 2$.

Оказывается, что всё это мы сделали за полиномиальное время по N, но ведь на самом деле мы решили SAT. В полученном списке формул будут лишь истина и ложь, так как мы подставили все значения переменных. Тогда просто проверяем, есть ли среди них истина. □

Замечание 4.2. *Унарный* NP*-трудный язык можно построить, например, с помощью функции трудолюбия Радо.*

5 Полиномиальная иерархия

Вспомним пройденное и адаптируем под новые реалии:

$$\Sigma_k^p: \ x \in A \iff \exists y_1 \forall y_2 \dots Q_k y_k \ V(x, y_1, \dots, y_k) = 1$$
$$\Pi_k^p: \ x \in A \iff \forall y_1 \exists y_2 \dots Q_k y_k \ V(x, y_1, \dots, y_k) = 1$$

V вычислим за полиномиальное время; $|y_i| \le p(|x|)$, где p — полином.

$$\Sigma_0^p = \Pi_0^p = \mathrm{P}; \quad \Sigma_1^p = \mathrm{NP}; \quad \Pi_1^p = \mathrm{coNP}$$

$$\Sigma_k^p \cup \Pi_k^p \subseteq \Sigma_{k+1}^p \cap \Pi_{k+1}^p; \quad \mathrm{PH} = \bigcup_{k \ge 0} \Sigma_k^p = \bigcup_{k \ge 0} \Pi_k^p$$

6 Дисклеймер для преподавателей перед любой сложной темой

